

Package ‘BvCh’

October 1, 2026

Type Package

Title Bivariate Chen Distribution: Density, Survival, Hazard, Plots
and Estimates

Version 0.1.0

Description Probability density, survival and hazard functions, random generation, graphical displays and maximum likelihood estimation for bivariate Chen type models. Three constructions are covered: the Marshall-Olkin type Bivariate Chen distribution with a singular component on the diagonal, the Bivariate Extended Chen distribution, and the Bivariate Inverse FGM Chen distribution obtained through the Inverse FGM copula with Chen marginals. Univariate Chen density, distribution, quantile, survival, hazard and random generation routines are obtained from the 'reliaR' package.

License GPL-3

Encoding UTF-8

Depends R (>= 4.0.0)

Imports reliaR (>= 0.2), stats, graphics, grDevices

Suggests testthat (>= 3.0.0), knitr, rmarkdown

VignetteBuilder knitr

URL <https://github.com/puneetstat999/BvCh>

BugReports <https://github.com/puneetstat999/BvCh/issues>

Config/roxygen2/version 8.1.0

NeedsCompilation no

Author Mukul Bijalwan [aut] (Orchid ID: <https://orcid.org/0009-0001-3040-6912>),
Puneet Kumar Gupta [aut, cre]

Maintainer Puneet Kumar Gupta <puneetstat999@gmail.com>

Contents

BvCh-package	2
bec	3
bvch	4
bvch_components	6
chen	7

compare_models	8
extendedchen	9
fitbec	10
fitbvch	11
fitifgmchen	12
hbec_grad	13
ifgm	14
ifgmchen	15
ksfit_bvch	17
loglik	17
plotbvch	18
print.fitbvch	21
Index	24

BvCh-package	<i>Bivariate Chen Distribution: Density, Survival, Hazard, Plots and Estimates</i>
--------------	--

Description

Implements probability density, survival and hazard functions, random generation, graphical displays and maximum likelihood estimation for a family of bivariate Chen type lifetime models.

Models

Bivariate Chen (BvCh) Marshall-Olkin type model of Gupta, Kumar and co-authors. With $U_i \sim \text{Chen}(\alpha_i, \beta)$ independent and $Z_1 = \min(U_1, U_3)$, $Z_2 = \min(U_2, U_3)$, the pair (Z_1, Z_2) is $BvCh(\alpha_1, \alpha_2, \alpha_3, \beta)$. The distribution is a mixture of an absolutely continuous part of total mass $(\alpha_1 + \alpha_2)/(\alpha_1 + \alpha_2 + \alpha_3)$ and a singular part of mass $\alpha_3/(\alpha_1 + \alpha_2 + \alpha_3)$ carried on the diagonal.

Bivariate Extended Chen (BEC) Marshall-Olkin type model using extended Chen latent variables, adding a shape parameter $\lambda > 0$.

Bivariate Inverse FGM Chen (IFGM-Chen) Copula construction pairing the Inverse FGM copula with Chen marginals. This model is absolutely continuous and has no singular component.

Univariate Chen

All univariate Chen building blocks are taken from the **reliaR** package rather than being re-implemented. In particular `reliaR::schen(x, beta, lambda)` satisfies `schen(x, beta, lambda) == exp(lambda * (1 - exp(x^beta)))`, which is the $S_{Ch}(z; \alpha, \beta)$ used throughout the bivariate formulae.

Author(s)

Maintainer: Puneet Kumar Gupta <puneetstat999@gmail.com>

Authors:

- Puneet Kumar Gupta <puneetstat999@gmail.com>
- Mukul Bijalwan (Orchid ID: <https://orcid.org/0009-0001-3040-6912>)

See Also

Useful links:

- <https://github.com/puneetstat999/BvCh>
- Report bugs at <https://github.com/puneetstat999/BvCh/issues>

bec	<i>Bivariate Extended Chen distribution: density, survival, hazard and random generation</i>
-----	--

Description

Probability density, survival function, hazard function and random generation for the Marshall-Olkin type Bivariate Extended Chen distribution $BEC(\alpha_1, \alpha_2, \alpha_3, \beta, \lambda)$.

Usage

```
dbec(x1, x2, alpha1, alpha2, alpha3, beta, lambda, log = FALSE)
sbec(x1, x2, alpha1, alpha2, alpha3, beta, lambda)
hbec(x1, x2, alpha1, alpha2, alpha3, beta, lambda)
rbec(n, alpha1, alpha2, alpha3, beta, lambda)
```

Arguments

x1, x2 numeric vectors of first and second component.
alpha1, alpha2, alpha3, beta, lambda
 positive parameters.
log logical; if TRUE the log density is returned.
n number of observations.

Details

With $U_i \sim EC(\alpha_i, \beta, \lambda)$ independent and $X_1 = \min(U_1, U_3)$, $X_2 = \min(U_2, U_3)$, the joint survival function is

$$S(x_1, x_2) = \begin{cases} S_{EC}(x_1; \alpha_1, \beta, \lambda) S_{EC}(x_2; \alpha_2 + \alpha_3, \beta, \lambda) & x_2 > x_1 \\ S_{EC}(x_1; \alpha_1 + \alpha_3, \beta, \lambda) S_{EC}(x_2; \alpha_2, \beta, \lambda) & x_1 > x_2 \\ S_{EC}(x; \alpha_1 + \alpha_2 + \alpha_3, \beta, \lambda) & x_1 = x_2 = x, \end{cases}$$

where $S_{EC}(t; \alpha, \beta, \lambda) = [1 + \lambda(e^{t^\beta} - 1)]^{-\alpha}$ (see [sech](#)). The joint density is

$$f(x_1, x_2) = \begin{cases} f_1 & x_2 > x_1 \\ f_2 & x_1 > x_2 \\ f_3 & x_1 = x_2 = x, \end{cases}$$

with

$$f_1 = \alpha_1(\alpha_2 + \alpha_3)\lambda^2\beta^2x_1^{\beta-1}x_2^{\beta-1}e^{x_1^\beta+x_2^\beta}[1 + \lambda(e^{x_1^\beta} - 1)]^{-(\alpha_1+1)}[1 + \lambda(e^{x_2^\beta} - 1)]^{-(\alpha_2+\alpha_3+1)},$$

$$f_2 = \alpha_2(\alpha_1 + \alpha_3)\lambda^2\beta^2x_1^{\beta-1}x_2^{\beta-1}e^{x_1^\beta+x_2^\beta}[1 + \lambda(e^{x_1^\beta} - 1)]^{-(\alpha_1+\alpha_3+1)}[1 + \lambda(e^{x_2^\beta} - 1)]^{-(\alpha_2+1)},$$

$$f_3(x) = \alpha_3\beta\lambda x^{\beta-1}e^{x^\beta}[1 + \lambda(e^{x^\beta} - 1)]^{-(\alpha_1+\alpha_2+\alpha_3+1)}.$$

The three regions carry mass $\alpha_i/(\alpha_1 + \alpha_2 + \alpha_3)$, so the density integrates to one. The hazard function is f/S ,

$$h(x_1, x_2) = \begin{cases} \frac{\alpha_1(\alpha_2 + \alpha_3)\lambda^2\beta^2x_1^{\beta-1}x_2^{\beta-1}e^{x_1^\beta+x_2^\beta}}{[1 + \lambda(e^{x_1^\beta} - 1)][1 + \lambda(e^{x_2^\beta} - 1)]} & x_2 > x_1 \\ \frac{\alpha_2(\alpha_1 + \alpha_3)\lambda^2\beta^2x_1^{\beta-1}x_2^{\beta-1}e^{x_1^\beta+x_2^\beta}}{[1 + \lambda(e^{x_1^\beta} - 1)][1 + \lambda(e^{x_2^\beta} - 1)]} & x_1 > x_2 \\ \frac{\alpha_3\beta\lambda x^{\beta-1}e^{x^\beta}}{1 + \lambda(e^{x^\beta} - 1)} & x_1 = x_2 = x. \end{cases}$$

The componentwise hazard gradients are available separately through [hbec1](#) and [hbec2](#).

Value

A numeric vector, or for rbec a two column matrix with columns x1 and x2.

See Also

[sbec](#), [hbec](#), [rbec](#), [fitbec](#), [sech](#)

Examples

```
dbec(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8, lambda = 2)
sbec(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8, lambda = 2)
hbec(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8, lambda = 2)
set.seed(1)
rbec(5, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8, lambda = 2)
```

bvch

Bivariate Chen distribution: density, survival, hazard and random generation

Description

Probability density, survival function, hazard function and random generation for the Marshall-Olkin type Bivariate Chen distribution $BvCh(\alpha_1, \alpha_2, \alpha_3, \beta)$.

Usage

```
dbvch(z1, z2, alpha1, alpha2, alpha3, beta, log = FALSE)

sbvch(z1, z2, alpha1, alpha2, alpha3, beta)

hbvch(z1, z2, alpha1, alpha2, alpha3, beta)

rbvch(n, alpha1, alpha2, alpha3, beta)
```

Arguments

z1, z2 numeric vectors of first and second component.
alpha1, alpha2, alpha3, beta
 positive parameters.
log logical; if TRUE the log density is returned.
n number of observations.

Details

Let $U_i \sim Chen(\alpha_i, \beta)$, $i = 1, 2, 3$, be independent and set $Z_1 = \min(U_1, U_3)$, $Z_2 = \min(U_2, U_3)$. Then the joint survival function is

$$S(z_1, z_2) = \exp \left\{ \alpha_1 \left(1 - e^{-z_1^\beta} \right) \right\} \exp \left\{ \alpha_2 \left(1 - e^{-z_2^\beta} \right) \right\} \exp \left\{ \alpha_3 \left(1 - e^{-w^\beta} \right) \right\},$$

where $w = \max(z_1, z_2)$. Each factor is obtained from [schen](#). The joint density is piecewise,

$$f(z_1, z_2) = \begin{cases} f_1(z_1, z_2) & z_1 < z_2 \\ f_2(z_1, z_2) & z_1 > z_2 \\ f_0(z) & z_1 = z_2 = z, \end{cases}$$

with

$$\begin{aligned} f_1 &= \alpha_1(\alpha_2 + \alpha_3)\beta^2 z_1^{\beta-1} z_2^{\beta-1} \exp \left\{ z_1^\beta + z_2^\beta + \alpha_1(1 - e^{-z_1^\beta}) + (\alpha_2 + \alpha_3)(1 - e^{-z_2^\beta}) \right\}, \\ f_2 &= \alpha_2(\alpha_1 + \alpha_3)\beta^2 z_1^{\beta-1} z_2^{\beta-1} \exp \left\{ z_1^\beta + z_2^\beta + \alpha_2(1 - e^{-z_2^\beta}) + (\alpha_1 + \alpha_3)(1 - e^{-z_1^\beta}) \right\}, \\ f_0(z) &= \alpha_3\beta z^{\beta-1} \exp \left\{ z^\beta + (\alpha_1 + \alpha_2 + \alpha_3)(1 - e^{-z^\beta}) \right\}. \end{aligned}$$

The mass carried by the three regions is respectively $\alpha_1/(\alpha_1 + \alpha_2 + \alpha_3)$, $\alpha_2/(\alpha_1 + \alpha_2 + \alpha_3)$ and $\alpha_3/(\alpha_1 + \alpha_2 + \alpha_3)$, so the density integrates to one. The hazard function is the ratio f/S , that is

$$h(z_1, z_2) = \begin{cases} \alpha_1(\alpha_2 + \alpha_3)\beta^2 z_1^{\beta-1} z_2^{\beta-1} e^{z_1^\beta + z_2^\beta} & z_1 < z_2 \\ \alpha_2(\alpha_1 + \alpha_3)\beta^2 z_1^{\beta-1} z_2^{\beta-1} e^{z_1^\beta + z_2^\beta} & z_1 > z_2 \\ \alpha_3\beta z^{\beta-1} e^{z^\beta} & z_1 = z_2 = z. \end{cases}$$

The model is a mixture (Proposition 1) of an absolutely continuous part of weight $(\alpha_1 + \alpha_2)/(\alpha_1 + \alpha_2 + \alpha_3)$ and a singular part of weight $\alpha_3/(\alpha_1 + \alpha_2 + \alpha_3)$ supported on the diagonal; see [dbvch_ac](#) and [dbvch_sing](#).

Value

A numeric vector, or for `rbvch` a two column matrix with columns `z1` and `z2`.

See Also

[sbvch](#), [hbvch](#), [rbvch](#), [fitbvch](#), [plot_surface_bvch](#)

Examples

```
dbvch(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
sbvch(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
hbvch(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
set.seed(1)
rbvch(5, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
```

bvch_components	<i>Absolutely continuous and singular components of the BvCh density</i>
-----------------	--

Description

Decomposition of the Bivariate Chen density into the two components of the mixture representation

$$f = \frac{\alpha_1 + \alpha_2}{\alpha_1 + \alpha_2 + \alpha_3} f_a + \frac{\alpha_3}{\alpha_1 + \alpha_2 + \alpha_3} f_s.$$

Usage

```
dbvch_ac(z1, z2, alpha1, alpha2, alpha3, beta)
```

```
dbvch_sing(z1, z2, alpha1, alpha2, alpha3, beta)
```

Arguments

z1, z2 numeric vectors of first and second component.
alpha1, alpha2, alpha3, beta
 positive parameters.

Details

dbvch_ac is the properly normalised absolutely continuous component of Proposition 1; it integrates to one over the plane, and dbvch_sing integrates to one along the diagonal. Consequently dbvch is recovered exactly as (alpha1 + alpha2)/(alpha1 + alpha2 + alpha3) * dbvch_ac(...) plus alpha3/(alpha1 + alpha2 + alpha3) * dbvch_sing(...).

Value

A numeric vector.

See Also

[dbvch](#)

Examples

```
a <- 1.5; b2 <- 2.5; c3 <- 0.5; bt <- 1.8
zz1 <- c(0.4, 0.9, 0.7); zz2 <- c(0.6, 0.4, 0.7)
ac <- dbvch_ac(zz1, zz2, a, b2, c3, bt)
sg <- dbvch_sing(zz1, zz2, a, b2, c3, bt)
k <- (a + b2 + c3)
mix <- (a + b2) / k * ac + c3 / k * sg
all.equal(mix, dbvch(zz1, zz2, a, b2, c3, bt))
```

chen	<i>Univariate Chen distribution</i>
------	-------------------------------------

Description

Density, distribution function, quantile function, random generation, survival and hazard function for the univariate Chen distribution with shape parameter beta and scale parameter lambda.

Usage

```
dchen(x, beta, lambda, log = FALSE)

pchen(q, beta, lambda, lower.tail = TRUE, log.p = FALSE)

qchen(p, beta, lambda, lower.tail = TRUE, log.p = FALSE)

rchen(n, beta, lambda)

schen(x, beta, lambda)

hchen(x, beta, lambda)

hra.chen(x, beta, lambda)

crf.chen(x, t = 0, beta, lambda)
```

Arguments

x, q	vector of quantiles.
beta	shape parameter.
lambda	scale parameter.
log, log.p	logical; if TRUE probabilities are returned as log(p).
lower.tail	logical; if TRUE (default) P[X <= x], otherwise P[X > x].
p	vector of probabilities.
n	number of observations.
t	age component for the conditional reliability function.

Details

These are the **reliaR** implementations, re-exported by **BvCh** so that the Chen building blocks used in the bivariate formulae are available without attaching a second package. The density is

$$f(x; \lambda, \beta) = \lambda \beta x^{\beta-1} \exp(x^\beta) \exp[\lambda \{1 - \exp(x^\beta)\}],$$

and the survival function is

$$S(x; \lambda, \beta) = \exp[\lambda \{1 - \exp(x^\beta)\}].$$

Taking $\lambda = \alpha$ gives the $S_{Ch}(z; \alpha, \beta)$ that appears in the bivariate Chen formulae.

Value

dchen gives the density, pchen the distribution function, qchen the quantile function, rchen random deviates, schen the survival function and hchen the hazard function.

Random generation

rchen is *not* a direct call to [rchen](#). In **reliaR** 0.2 that function does not agree with the package's own distribution, quantile and survival functions: for $\beta = 1.5$, $\lambda = 2$ its output has Kolmogorov-Smirnov distance about 0.38 from pchen ($p < 10^{-16}$), and the discrepancy grows with β (distance about 0.85 at $\beta = 2.5$); it happens to be correct only when $\beta = 1$. Since qchen is the verified analytic inverse of pchen (agreement to machine precision), rchen here draws qchen(runif(n), beta, lambda), which keeps all six univariate Chen routines mutually consistent. Everything is still computed by **reliaR**.

See Also

[Chen](#), [Chensurvival](#)

Examples

```
dchen(0.5, beta = 1.5, lambda = 2)
pchen(0.5, beta = 1.5, lambda = 2)
qchen(0.5, beta = 1.5, lambda = 2)
schen(0.5, beta = 1.5, lambda = 2)
hchen(0.5, beta = 1.5, lambda = 2)
```

compare_models	<i>Model comparison table</i>
----------------	-------------------------------

Description

Collects fitted models into a single table of $-\log L$, AIC and BIC.

Usage

```
compare_models(...)
```

Arguments

... fitted model objects from [fitbvch](#), [fitbec](#) or [fitifgmchen](#).

Value

A data frame with columns Model, nobs, neglogL, AIC and BIC, sorted by increasing AIC so that the best model is on the first row.

Examples

```
set.seed(1)
a <- fitbvch(rbvch(200, 1.5, 2.5, 0.5, 1.8))
b <- fitbec(rbec(200, 1.5, 2.5, 0.5, 1.8, 2))
compare_models(a, b)
```

extendedchen

*Univariate Extended Chen distribution***Description**

Density, distribution function, quantile function, random generation, survival and hazard function for the univariate Extended Chen distribution used as the latent building block of the Bivariate Extended Chen (BEC) model.

Usage

```
sech(x, alpha, beta, lambda)
hech(x, alpha, beta, lambda)
dech(x, alpha, beta, lambda, log = FALSE)
pech(q, alpha, beta, lambda, lower.tail = TRUE, log.p = FALSE)
qech(p, alpha, beta, lambda, lower.tail = TRUE, log.p = FALSE)
rech(n, alpha, beta, lambda)
```

Arguments

x, q	vector of quantiles.
alpha, lambda, beta	positive parameters.
log, log.p	logical; if TRUE probabilities are returned as log(p).
lower.tail	logical; if TRUE (default) $P[X \leq x]$, otherwise $P[X > x]$.
p	vector of probabilities.
n	number of observations.

Details

The Extended Chen survival function is

$$S_{EC}(x; \alpha, \beta, \lambda) = [1 + \lambda (\exp(x^\beta) - 1)]^{-\alpha},$$

giving hazard

$$h_{EC}(x; \alpha, \beta, \lambda) = \frac{\alpha \lambda \beta x^{\beta-1} \exp(x^\beta)}{1 + \lambda (\exp(x^\beta) - 1)}$$

and density $f_{EC}(x) = S_{EC}(x) h_{EC}(x)$. The quantile function is available in closed form,

$$Q(p) = \left[\log \left\{ 1 + \frac{p^{-1/\alpha} - 1}{\lambda} \right\} \right]^{1/\beta}.$$

Unlike the univariate Chen distribution, which **reliaR** supplies, no Extended Chen implementation exists there, so it is provided here.

Value

dech gives the density, pech the distribution function, qech the quantile function, rech random deviates, sech the survival function and hech the hazard function.

Examples

```
sech(0.5, alpha = 1.5, beta = 1.5, lambda = 2)
hech(0.5, alpha = 1.5, beta = 1.5, lambda = 2)
qech(c(0.25, 0.5, 0.75), alpha = 1.5, beta = 1.5, lambda = 2)
```

fitbec

*Maximum likelihood estimation for the BEC model***Description**

Fits the Bivariate Extended Chen model by maximum likelihood.

Usage

```
fitbec(
  x1,
  x2 = NULL,
  start = NULL,
  singular = TRUE,
  method = "BFGS",
  control = list(),
  se = TRUE
)
```

Arguments

x1, x2	numeric vectors of observations, or a two column matrix.
start	optional starting values on the natural scale in the order alpha1, alpha2, alpha3, beta, lambda.
singular	logical; whether tied observations contribute the singular density.
method	optimisation method passed to optim .
control	optional control list for optim .
se	logical; whether standard errors should be computed.

Value

An object of class `fitbec` with the same components as [fitbvch](#).

See Also

[fitbvch](#), [logLik_bec](#)

Examples

```
set.seed(1)
d <- rbec(400, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8,
          lambda = 2)
fit <- fitbec(d)
fit
```

fitbvch

Maximum likelihood estimation for the BvCh model

Description

Fits the Marshall-Olkin type Bivariate Chen model by maximum likelihood.

Usage

```
fitbvch(
  x1,
  x2 = NULL,
  start = NULL,
  singular = TRUE,
  method = "BFGS",
  control = list(),
  se = TRUE
)
```

Arguments

<code>x1, x2</code>	numeric vectors of observations, or a two column matrix.
<code>start</code>	optional numeric vector of starting values on the natural scale, in the order <code>alpha1</code> , <code>alpha2</code> , <code>alpha3</code> , <code>beta</code> .
<code>singular</code>	logical; whether tied observations contribute the singular density.
<code>method</code>	optimisation method passed to <code>optim</code> .
<code>control</code>	optional control list for <code>optim</code> .
<code>se</code>	logical; whether standard errors should be computed.

Details

Observations with $z_1 == z_2$ fall on the singular component of the model. Whether they contribute the singular density f_0 or are discarded is controlled by `singular`; the counts are always reported so the choice stays visible. Standard errors come from the numerical Hessian of the log-likelihood and are on the original (untransformed) scale.

Value

An object of class `fitbvch`: a list with components `coefficients`, `vcov`, `se`, `loglik`, `aic`, `bic`, `nobs`, `counts`, `converged`, `message` and `call`.

See Also

[fitbec](#), [fitifgmchen](#), [logLik_bvch](#)

Examples

```
set.seed(1)
d <- rlvch(400, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
fit <- fitbvch(d)
fit
coef(fit)
```

fitifgmchen

Maximum likelihood estimation for the IFGM-Chen model

Description

Fits the Bivariate Inverse FGM Chen model by maximum likelihood.

Usage

```
fitifgmchen(
  x1,
  x2 = NULL,
  start = NULL,
  method = "BFGS",
  control = list(),
  se = TRUE
)
```

Arguments

<code>x1, x2</code>	numeric vectors of observations, or a two column matrix.
<code>start</code>	optional starting values on the natural scale in the order <code>theta1</code> , <code>beta1</code> , <code>theta2</code> , <code>beta2</code> , <code>gamma</code> , <code>omega</code> .
<code>method</code>	optimisation method passed to optim .
<code>control</code>	optional control list for optim .
<code>se</code>	logical; whether standard errors should be computed.

Details

The copula parameters `gamma` and `omega` are constrained to $[0, 1]$ and are estimated on the logit scale; the remaining parameters are estimated on the log scale. All reported values are on the natural scale.

Value

An object of class `fitifgmchen` with the same components as [fitbvch](#).

Identifiability

The four marginal parameters are estimated very precisely, but gamma and omega are only weakly identified: with $n = 5000$ simulated from $(\gamma, \omega) = (0.6, 0.4)$ the marginal parameters have standard errors around 0.014 while gamma and omega have standard errors around 0.54 and 1.29. The log-likelihood surface is nearly flat in that direction, so individual values of gamma and omega should not be over-interpreted; logLik_ifgmchen is exported so the profile likelihood can be inspected and start used to explore it.

See Also

[fitbvch](#), [logLik_ifgmchen](#)

Examples

```
set.seed(1)
d <- rifgmchen(400, theta1 = 1.2, beta1 = 1.3, theta2 = 1.5,
               beta2 = 1.1, gamma = 0.6, omega = 0.4)
fit <- fitifgmchen(d)
fit
```

hbec_grad

Componentwise hazard gradients of the BEC model

Description

The hazard gradients of the Bivariate Extended Chen model, that is the hazard of the i -th marginal conditional on the other component.

Usage

hbec1(x1, x2, alpha1, alpha2, alpha3, beta, lambda)

hbec2(x1, x2, alpha1, alpha2, alpha3, beta, lambda)

Arguments

x1, x2 numeric vectors of first and second component.
alpha1, alpha2, alpha3, beta, lambda
 positive parameters.

Details

$$h_{X_1}(x_1, x_2) = \begin{cases} \frac{\alpha_1 \beta \lambda x_1^{\beta-1} e^{x_1^\beta}}{1 + \lambda(e^{x_1^\beta} - 1)} & x_2 > x_1 \\ \frac{(\alpha_1 + \alpha_3) \beta \lambda x_1^{\beta-1} e^{x_1^\beta}}{1 + \lambda(e^{x_1^\beta} - 1)} & x_1 > x_2 \\ \frac{(\alpha_1 + \alpha_2 + \alpha_3) \beta \lambda x_1^{\beta-1} e^{x_1^\beta}}{1 + \lambda(e^{x_1^\beta} - 1)} & x_1 = x_2, \end{cases}$$

with h_{X_2} obtained by interchanging the roles of x_1, x_2 and α_1, α_2 .

Value

A numeric vector.

See Also

[hbec](#)

Examples

```
hbec1(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8, lambda = 2)
hbec2(0.5, 0.9, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8, lambda = 2)
```

ifgm	<i>Inverse FGM copula</i>
------	---------------------------

Description

The Inverse FGM (IFGM) copula, its density, dependence measures and a random generator.

Usage

```
cifgm(u, v, gamma, omega)
cifgmdens(u, v, gamma, omega)
rifgm(n, gamma, omega)
tau_ifgm(gamma, omega)
rho_ifgm(gamma, omega)
```

Arguments

`u, v` numeric vectors of copula arguments in $[0, 1]$.
`gamma, omega` copula parameters, each in $[0, 1]$.
`n` number of observations.

Details

For $0 \leq \gamma, \omega \leq 1$ the Inverse FGM copula is

$$C(u, v) = \frac{uv}{\sqrt{1 + \gamma(1-u)(1-v) + \omega uv(1-u)(1-v)}}.$$

Writing $D = 1 + \gamma(1-u)(1-v) + \omega uv(1-u)(1-v)$ and $P = uv$, its density has the closed form

$$c(u, v) = D^{-1/2} - \frac{vD_v + uD_u}{2D^{3/2}} + \frac{3PD_uD_v}{4D^{5/2}} - \frac{PD_{uv}}{2D^{3/2}},$$

with $D_u = -\gamma(1-v) + \omega(1-2u)v(1-v)$, $D_v = -\gamma(1-u) + \omega u(1-2v)(1-u)$ and $D_{uv} = \gamma + \omega(1-2u)(1-2v)$.

Kendall's tau and Spearman's rho are computed as

$$\tau = 4 \int_0^1 \int_0^1 C(u, v) c(u, v) du dv - 1, \quad \rho = 12 \int_0^1 \int_0^1 uv c(u, v) du dv - 3.$$

Both are obtained by numerical quadrature and reproduce Monte Carlo estimates from `rifgm`.

Value

`cifgm` gives the copula, `cifgmdens` its density, `rifgm` a two column matrix of copula draws, and `tau_ifgm` and `rho_ifgm` scalar dependence measures.

Sign of the dependence

With the $+\gamma(1-u)(1-v) + \omega uv(1-u)(1-v)$ denominator given above, the copula lies *below* the independence copula uv ($C(0.5, 0.5) = 0.2306 < 0.25$ at $\gamma = 0.6, \omega = 0.4$), so τ and ρ are non-positive for $\gamma, \omega \geq 0$: numerically $\tau = -0.0699$ and $\rho = -0.1049$ at $\gamma = 0.6, \omega = 0.4$. This is the usual behaviour of an *inverse* FGM type copula. Both measures are exactly zero at $\gamma = \omega = 0$. Note that the summary document this package was written from quotes $\tau \approx 0.156 > 0$ for the same parameters, which is inconsistent with the copula formula it also quotes; the formula has been implemented as given, and `tau_ifgm`/`rho_ifgm` are provided so the dependence can be checked directly.

See Also

[difgmchen](#), [hifgmchen](#)

Examples

```
cifgm(0.3, 0.7, gamma = 0.6, omega = 0.4)
cifgmdens(0.3, 0.7, gamma = 0.6, omega = 0.4)
tau_ifgm(0.6, 0.4)
rho_ifgm(0.6, 0.4)
set.seed(1)
head(rifgm(5, gamma = 0.6, omega = 0.4))
```

ifgmchen

Bivariate IFGM-Chen distribution

Description

Reliability function, density, hazard function and random generation for the Bivariate Inverse FGM Chen model, obtained by combining the Inverse FGM copula with Chen marginals.

Usage

```
relifgmchen(x, y, theta1, beta1, theta2, beta2, gamma, omega)

difgmchen(x, y, theta1, beta1, theta2, beta2, gamma, omega, log = FALSE)

hifgmchen(x, y, theta1, beta1, theta2, beta2, gamma, omega)

rifgmchen(n, theta1, beta1, theta2, beta2, gamma, omega)
```

Arguments

<code>x, y</code>	numeric vectors of first and second component.
<code>theta1, beta1, theta2, beta2</code>	positive marginal parameters.
<code>gamma, omega</code>	copula parameters in $[0, 1]$.
<code>log</code>	logical; if TRUE the log density is returned.
<code>n</code>	number of observations.

Details

The marginals are Chen distributions $F_X(x) = 1 - e^{-\theta_1(e^{x^{\beta_1}} - 1)}$ and $F_Y(y) = 1 - e^{-\theta_2(e^{y^{\beta_2}} - 1)}$, which are exactly `reliaR::pchen(x, beta1, theta1)` and `reliaR::pchen(y, beta2, theta2)`. With C the Inverse FGM copula of `cifgm` and c its density, the joint reliability function and density are

$$R(x, y) = C(F_X(x), F_Y(y)) - F_X(x) - F_Y(y) + 1,$$

$$f(x, y) = c(F_X(x), F_Y(y)) f_X(x) f_Y(y),$$

where $f_X = [\text{reliaR}]dchen(x, \beta_1, \theta_1)$ and $f_Y = [\text{reliaR}]dchen(y, \beta_2, \theta_2)$. The hazard function is the ratio f/R .

Value

A numeric vector, or for `rifgmchen` a two column matrix with columns `x` and `y`.

Remarks

Unlike the Marshall-Olkin type models, this construction is absolutely continuous: it has no mass on the diagonal and therefore no singular component. The survival function is $1 - \text{pchen}(x, \text{beta1}, \text{theta1}) - \text{pchen}(y, \text{beta2}, \text{theta2}) + R(x, y)$ and can be negative by cancellation in the far tail.

See Also

`cifgm`, `fitifgmchen`, `plot_reliability_ifgmchen`

Examples

```
difgmchen(0.5, 0.9, theta1 = 1.2, beta1 = 1.3, theta2 = 1.5,
           beta2 = 1.1, gamma = 0.6, omega = 0.4)
relifgmchen(0.5, 0.9, theta1 = 1.2, beta1 = 1.3, theta2 = 1.5,
            beta2 = 1.1, gamma = 0.6, omega = 0.4)
hifgmchen(0.5, 0.9, theta1 = 1.2, beta1 = 1.3, theta2 = 1.5,
          beta2 = 1.1, gamma = 0.6, omega = 0.4)
set.seed(1)
head(rifgmchen(5, theta1 = 1.2, beta1 = 1.3, theta2 = 1.5,
              beta2 = 1.1, gamma = 0.6, omega = 0.4))
```

ksfit_bvch

*Kolmogorov-Smirnov goodness of fit for the BvCh marginals***Description**

Tests the fitted marginals of a Bivariate Chen fit against the data using [ks.chen](#).

Usage

```
ksfit_bvch(fit, x1, x2 = NULL)
```

Arguments

`fit` a fitted model from [fitbvch](#).
`x1, x2` numeric vectors of observations, or a two column matrix.

Details

The two components are compared with $Chen(\alpha_1 + \alpha_3, \beta)$ and $Chen(\alpha_2 + \alpha_3, \beta)$, since $Z_1 = \min(U_1, U_3)$ and $Z_2 = \min(U_2, U_3)$, and their minimum with $Chen(\alpha_1 + \alpha_2 + \alpha_3, \beta)$, which is the marginal of $\min(Z_1, Z_2) = \min(U_1, U_2, U_3)$. Each of these is a Chen distribution with the same shape β , because the minimum of independent Chen variables is Chen in the summed scale parameter.

Value

A data frame with one row per series (Z_1 , Z_2 and $\min(Z_1, Z_2)$), giving the fitted alpha and beta, the Kolmogorov-Smirnov distance D and its p-value $p.value$, together with the sample mean and standard deviation.

Examples

```
set.seed(1)
d <- rrvch(200, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
fit <- fitbvch(d)
ksfit_bvch(fit, d)
```

loglik

*Log-likelihood functions of the bivariate Chen models***Description**

Log-likelihoods used by [fitbvch](#), [fitbec](#) and [fitifgmchen](#).

Usage

```
logLik_bvch(par, z1, z2, singular = TRUE)

logLik_bec(par, z1, z2, singular = TRUE)

logLik_ifgmchen(par, z1, z2)
```

Arguments

par	numeric vector of parameters on the log scale.
z1, z2	numeric vectors of observations.
singular	logical; if TRUE ties ($z_1 == z_2$) contribute the singular density. Use FALSE to drop tied observations.

Details

For the Marshall-Olkin type models the log-likelihood is the exact observed-data log-likelihood: the contribution of an observation is entirely determined by whether $z_1 < z_2$, $z_1 > z_2$ or $z_1 = z_2$, so the complete-data log-likelihood coincides with the observed-data one. Direct numerical maximisation is therefore used; it reaches the same fixed point as the EM algorithm.

Value

The sum of the log densities, or $-\text{Inf}$ when any contribution is not finite.

Examples

```
set.seed(1)
d <- rrvch(50, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
logLik_bvch(log(c(1.5, 2.5, 0.5, 1.8)), d[, 1], d[, 2])
```

plotbvch

Surface and contour plots for the bivariate Chen models

Description

Graphical displays of the density, survival or reliability surfaces of the BvCh, BEC and IFGM-Chen models, using base R graphics only.

Usage

```
plot_surface_bvch(
  alpha1 = 1.5,
  alpha2 = 2.5,
  alpha3 = 0.5,
  beta.vec = c(0.8, 1.8, 5, 10),
  what = c("density", "survival", "hazard"),
  part = c("ac", "singular", "all"),
  n.grid = 61L,
  range = 3,
  theta = 30,
  phi = 25,
  expand.fac = 0.6,
  col.colours = NULL,
  n.col = 100,
  main = NULL,
  ...
)
```

```
plot_surface_bec(  
  alpha1 = 1.5,  
  alpha2 = 2.5,  
  alpha3 = 0.5,  
  beta = 1.8,  
  lambda = 2,  
  what = c("density", "survival", "hazard"),  
  n.grid = 61L,  
  range = 3,  
  theta = 30,  
  phi = 25,  
  expand.fac = 0.6,  
  col.colours = NULL,  
  n.col = 100,  
  main = NULL,  
  ...  
)
```

```
plot_reliability_ifgmchen(  
  theta1 = 1.2,  
  beta1 = 1.3,  
  theta2 = 1.5,  
  beta2 = 1.1,  
  gamma = 0.6,  
  omega = 0.4,  
  n.grid = 61L,  
  range = 3,  
  theta = 30,  
  phi = 25,  
  expand.fac = 0.6,  
  col.colours = NULL,  
  n.col = 100,  
  main = NULL,  
  ...  
)
```

```
plot_contour_bvch(  
  alpha1 = 1.5,  
  alpha2 = 2.5,  
  alpha3 = 0.5,  
  beta = 1.8,  
  n.grid = 81L,  
  range = 3,  
  n.levels = 12,  
  lab = NULL,  
  add = FALSE,  
  draw.singular = TRUE,  
  main = NULL,  
  ...  
)
```

Arguments

alpha1, alpha2, alpha3, beta, lambda	model parameters.
beta.vec	a vector of β values, one surface per value.
what	one of "density", "survival" or "hazard".
part	one of "ac", "singular" or "all"; only used when what = "density" for the BvCh model.
n.grid	resolution of the plotting grid in each direction.
range	maximum value of both axes.
theta, phi	camera angles passed to persp .
expand.fac	expansion factor for the z axis.
col.colours	colours for the surface: a vector of colour names, a function such as heat.colors , or NULL for the default heat palette.
n.col	number of colour bands.
main	optional plot title; a default is built from the parameters.
...	further arguments passed to persp .
theta1, beta1, theta2, beta2	IFGM-Chen marginal parameters.
gamma, omega	IFGM-Chen copula parameters.
lab	main title for the contour plot.
add	if TRUE the contour is drawn on the density surface.
draw.singular	logical; draw the singular support on the diagonal of a contour plot.

Details

`plot_surface_bvch` draws the absolutely continuous part of the Bivariate Chen density, which is the surface shown for varying β in the source paper: as β increases the surface changes from flat or monotone to peaked near the origin. `part` selects between the absolutely continuous part, the singular part and the full piecewise density.

Value

Invisibly, the matrix of values plotted. `plot_surface_bvch` returns one matrix per element of `beta.vec`.

See Also

[dbvch](#), [dbec](#), [difgmchen](#)

Examples

```
plot_surface_bvch(alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5,
                  beta.vec = c(0.8, 1.8, 5, 10))
plot_surface_bec(alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8,
                 lambda = 2)
plot_reliability_ifgmchen(theta1 = 1.2, beta1 = 1.3, theta2 = 1.5,
                          beta2 = 1.1, gamma = 0.6, omega = 0.4)
plot_contour_bvch(alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
```

print.fitbvch	<i>Print, summary and extractor methods for fitted bivariate Chen models</i>
---------------	--

Description

Methods for the objects returned by `fitbvch`, `fitbec` and `fitifgmchen`.

Usage

```
## S3 method for class 'fitbvch'
print(x, digits = 4, ...)

## S3 method for class 'fitbec'
print(x, digits = 4, ...)

## S3 method for class 'fitifgmchen'
print(x, digits = 4, ...)

## S3 method for class 'fitbvch'
summary(object, digits = 4, signif.stars = FALSE, ...)

## S3 method for class 'fitbec'
summary(object, digits = 4, signif.stars = FALSE, ...)

## S3 method for class 'fitifgmchen'
summary(object, digits = 4, signif.stars = FALSE, ...)

## S3 method for class 'fitbvch'
coef(object, ...)

## S3 method for class 'fitbec'
coef(object, ...)

## S3 method for class 'fitifgmchen'
coef(object, ...)

## S3 method for class 'fitbvch'
vcov(object, ...)

## S3 method for class 'fitbec'
vcov(object, ...)

## S3 method for class 'fitifgmchen'
vcov(object, ...)

## S3 method for class 'fitbvch'
nobs(object, ...)

## S3 method for class 'fitbec'
nobs(object, ...)
```

```

## S3 method for class 'fitifgmchen'
nobs(object, ...)

## S3 method for class 'fitbvch'
logLik(object, ...)

## S3 method for class 'fitbec'
logLik(object, ...)

## S3 method for class 'fitifgmchen'
logLik(object, ...)

## S3 method for class 'fitbvch'
print(x, digits = 4, ...)
## S3 method for class 'fitbec'
print(x, digits = 4, ...)
## S3 method for class 'fitifgmchen'
print(x, digits = 4, ...)
## S3 method for class 'fitbvch'
summary(object, digits = 4, signif.stars = FALSE, ...)
## S3 method for class 'fitbvch'
coef(object, ...)
## S3 method for class 'fitbec'
coef(object, ...)
## S3 method for class 'fitifgmchen'
coef(object, ...)
## S3 method for class 'fitbvch'
vcov(object, ...)
## S3 method for class 'fitbec'
vcov(object, ...)
## S3 method for class 'fitifgmchen'
vcov(object, ...)
## S3 method for class 'fitbvch'
nobs(object, ...)
## S3 method for class 'fitbec'
nobs(object, ...)
## S3 method for class 'fitifgmchen'
nobs(object, ...)
## S3 method for class 'fitbvch'
logLik(object, ...)
## S3 method for class 'fitbec'
logLik(object, ...)
## S3 method for class 'fitifgmchen'
logLik(object, ...)

```

Arguments

<code>x, object</code>	a fitted model object.
<code>digits</code>	number of significant digits to print.
<code>...</code>	further arguments, currently unused.
<code>signif.stars</code>	logical; whether to print significance stars.

Details

The fitted objects also carry `aic` and `bic` components, and [compare_models](#) tabulates them across models. Because `aic` and `bic` are not S3 generics that a package can register methods for, no `aic` or `bic` methods are provided.

Value

The printing methods return their argument invisibly; `coef`, `vcov`, `nobs` and `logLik` return the corresponding component of the fit.

Examples

```
set.seed(1)
d <- rlvch(300, alpha1 = 1.5, alpha2 = 2.5, alpha3 = 0.5, beta = 1.8)
fit <- fitbvch(d)
print(fit)
summary(fit)
coef(fit)
vcov(fit)
nobs(fit)
```

Index

* internal

BvCh-package, 2
plotbvch, 18

[, 16

bec, 3

BvCh (BvCh-package), 2

bvch, 4

BvCh-package, 2

bvch_components, 6

Chen, 8

chen, 7

Chensurvival, 8

cifgm, 16

cifgm (ifgm), 14

cifgmdens (ifgm), 14

coef.fitbec (print.fitbvch), 21

coef.fitbvch (print.fitbvch), 21

coef.fitifgmchen (print.fitbvch), 21

compare_models, 8, 23

crf.chen (chen), 7

dbec, 20

dbec (bec), 3

dbvch, 6, 20

dbvch (bvch), 4

dbvch_ac, 5

dbvch_ac (bvch_components), 6

dbvch_sing, 5

dbvch_sing (bvch_components), 6

dchen (chen), 7

dech (extendedchen), 9

difgmchen, 15, 20

difgmchen (ifgmchen), 15

extendedchen, 9

fitbec, 4, 8, 10, 12, 17, 21

fitbvch, 5, 8, 10, 11, 12, 13, 17, 21

fitifgmchen, 8, 12, 12, 16, 17, 21

hbec, 4, 14

hbec (bec), 3

hbec1, 4

hbec1 (hbec_grad), 13

hbec2, 4

hbec2 (hbec_grad), 13

hbec_grad, 13

hbvch, 5

hbvch (bvch), 4

hchen (chen), 7

heat.colors, 20

hech (extendedchen), 9

hifgmchen, 15

hifgmchen (ifgmchen), 15

hra.chen (chen), 7

ifgm, 14

ifgmchen, 15

ks.chen, 17

ksfit_bvch, 17

loglik, 17

logLik.fitbec (print.fitbvch), 21

logLik.fitbvch (print.fitbvch), 21

logLik.fitifgmchen (print.fitbvch), 21

logLik_bec, 10

logLik_bec (loglik), 17

logLik_bvch, 12

logLik_bvch (loglik), 17

logLik_ifgmchen, 13

logLik_ifgmchen (loglik), 17

nobs.fitbec (print.fitbvch), 21

nobs.fitbvch (print.fitbvch), 21

nobs.fitifgmchen (print.fitbvch), 21

optim, 10–12

pchen (chen), 7

pech (extendedchen), 9

persp, 20

plot_contour_bvch (plotbvch), 18

plot_reliability_ifgmchen, 16

plot_reliability_ifgmchen (plotbvch), 18

plot_surface_bec (plotbvch), 18

plot_surface_bvch, 5

plot_surface_bvch (plotbvch), 18

plotbvch, 18
print.fitbec (print.fitbvch), 21
print.fitbvch, 21
print.fitifgmchen (print.fitbvch), 21

qchen (chen), 7
qech (extendedchen), 9

rbec, 4
rbec (bec), 3
rbvch, 5
rbvch (bvch), 4
rchen, 8
rchen (chen), 7
rech (extendedchen), 9
relifgmchen (ifgmchen), 15
rho_ifgm (ifgm), 14
rifgm (ifgm), 14
rifgmchen (ifgmchen), 15

sbec, 4
sbec (bec), 3
sbvch, 5
sbvch (bvch), 4
schen, 5
schen (chen), 7
sech, 3, 4
sech (extendedchen), 9
summary.fitbec (print.fitbvch), 21
summary.fitbvch (print.fitbvch), 21
summary.fitifgmchen (print.fitbvch), 21

tau_ifgm (ifgm), 14

vcov.fitbec (print.fitbvch), 21
vcov.fitbvch (print.fitbvch), 21
vcov.fitifgmchen (print.fitbvch), 21